서울시립대학교
UNIVERSITY OF SEOUL

전자전기컴퓨터설계실험II

실험 전 보고서

STM32와 FPGA의 SPI 송수신

학과	전자전기컴퓨터공학부
이름 / 학번	조성현 / 2023440129
작성일	2026년 11월 02일
대상	STM32 과제8 · STM32 과제9 · STM32 과제10 · STM32 과제11 · STM32 과제12
검증환경	Windows 11 / VS Code / STM32 CMake / Icarus Verilog / VaporView

소스 식별

GitHub 계정: a40093632-ai

GitHub 저장소: https://github.com/a40093632-ai/ece2-lab7

제출 태그: lab7-pre-v2

커밋 해시: 5a75763942e0c37c230f0d6eaf6bf10024c01c27

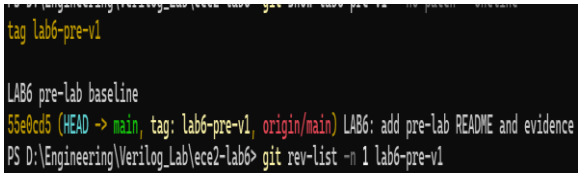


그림 1: GitHub 제출 태그 및 기준 커밋 확인

1. 실험 목적 및 범위

실험 목적

본 실험의 목적은 STM32와 FPGA 사이에 SPI 통신 경로를 구성하고, 송신 파형과 수신 데이터를 비교하는 것이다. MCU의 SPI 하드웨어를 이용한 송수신을 먼저 확인한 뒤, FPGA에서 SPI 송신·수신 회로를 직접 작성하여 같은 통신 조건으로 동작하도록 한다.

과제8에서는 MCU가 출력하는 SCK·MOSI·CS의 관계와 비트 순서를 확인한다. 과제9에서는 MCU 두 대를 연결하여 송신 값과 수신 MCU의 UART 출력을 비교한다. 과제10·11에서는 MCU와 FPGA의 역할을 바꾸어 통신하고, 과제12에서는 앞서 작성한 FPGA 송신 회로와 수신 회로를 연결한다.

실험 범위

	실습 내용	사전 검증 방법
과제8	MCU 송신 파형 확인	송신 CMake 빌드
과제9	MCU → MCU 수신	수신 CMake 빌드
과제 10	MCU → FPGA 수신	SPI 수신 및 UART 통합 시뮬레이션
과제 11	FPGA → MCU 송신	FPGA 송신 시뮬레이션, MCU 수신 코드 재사용
과제 12	FPGA → FPGA 송수신 확인	송수신 회로 연결 시뮬레이션

2. 관련 이론

SPI의 비트 전송과 시프트 레지스터

SPI는 마스터가 제공하는 클록에 맞추어 데이터를 전달하는 동기식 직렬 통신이다. SCK는 전송 시점을 정하는 클록, MOSI는 마스터에서 슬레이브로 전달되는 데이터, MISO는 반대 방향의 데이터 신호이다. CS는 통신할 슬레이브를 선택하며, 이번 실험에서는 LOW일 때 선택된다.

직렬 데이터를 병렬 데이터로 복원할 때는 시프트 레지스터를 사용할 수 있다. 입력 비트를 하나씩 받아 기존 내용을 이동시키면 여러 클록에 걸쳐 한 워드가 구성된다.[1] 이번 수신 회로는 MSB부터 들어오는 비트를 다음과 같이 저장한다.

shift_reg ← {shift_reg[6:0], MOSI}

8번째 비트까지 받은 뒤 완성된 바이트를 출력하고 수신 완료 신호를 발생시킨다. 예를 들어 0xA5는 상승 에지에서 읽은 비트가 1 0 1 0 0 1 0 1일 때 복원된다. CS가 중간에 해제되면 미완성 데이터를 폐기하고 다음 전송부터 다시 수신한다.

3. 개발 및 검증 환경

개발환경

항목	사용 환경
운영체제	Windows 11 64-bit
편집기	VS Code 1.137.0
파형 확인	VaporView
FPGA Part	Xc7s75fgga484-1
FPGA 도구	AMD Vivado 2026.1
보드	Combo II-DLD S75
STM32 빌드	CMake 기반 STM32
STM32 대상	NUCLEO-G474RE
UART 확인 조건	9600 baud, 8N1, Text 표시

주요 연결과 파일 역할

신호	MCU 마스터	MCU 슬레이브	FPGA XDC 지정
SCK	PB3	PB3	F8, BDIO0
MOSI	PA7	PA7	E8, BDIO1
CS / NSS	PB6	PA4	A7, BDIO2
GND	공통 접지	공통 접지	공통 접지

4. STM32 과제 8

설계 목표

MCU가 1초마다 한 바이트를 송신하도록 구성하고, 오실로스코프로 SCK·MOSI·CS의 관계를 확인한다. 송신값을 0x55, 0xA5, 0x00, 0xFF로 바꾸어 비트 순서와 클록 주파수를 비교한다.

핵심 설계

HAL_GetTick()으로 이전 송신 이후 1000 ms가 지났는지 확인한다. 송신 시 PB6의 CS를 LOW로 내리고 HAL_SPI_Transmit()으로 한 바이트를 전송한 뒤 CS를 HIGH로 복귀시킨다. 이후 송신값과 함수 반환값을 VCP UART에 출력한다. 기본 송신값은 0x55이다.

예상 결과

한 바이트의 전송 구간에는 SCK 상승 에지가 8번 나타나고, 전송 사이에는 SCK가 LOW, CS가 HIGH로 유지될 것으로 예상한다

송신값	읽을 비트 순서
0x55	01010101
0xA5	10100101
0x00	00000000
0xFF	11111111

사전 검증 결과

송신 코드의 컴파일·링크가 완료되었으며 빌드 종료 코드 0을 확인하였다. 실제 SCK 주파수와 MOSI의 비트 순서는 실험실에서 측정한다.

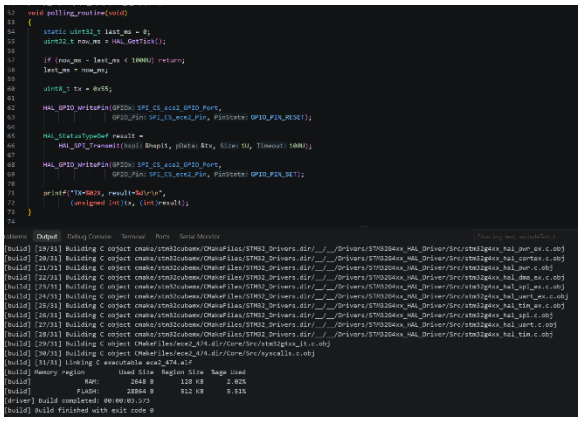


그림 2: STM32 과제 8 송신 코드 및 CMake 빌드 결과

링크: [과제8 소스 코드]

링크: [과제8 빌드 및 핵심 코드 증빙]

5. STM32 과제 9

설계 목표

지정된 조의 MCU 두 대를 연결하여 한쪽에서 보낸 바이트가 수신 측 VCP UART에 동일하게 출력되는지 확인한다. 마스터와 슬레이브 역할을 교대하여 송신·수신을 모두 수행한다.

핵심 설계

송신 측은 과제8의 마스터 프로그램을 사용한다. 수신 측은 ece2_474_init()에서 SPI1을 해제한 뒤 SPI_MODE_SLAVE와 SPI_NSS_HARD_INPUT으로 다시 초기화한다. PA4는 AF5의 SPI1 NSS 입력으로 설정하고 풀업을 적용한다.

수신 함수는 HAL_SPI_Receive()로 한 바이트를 기다린다. 정상 수신 시 RX=%02X 형식으로 출력하고, 2000 ms 동안 수신하지 못하면 타임아웃 메시지를 출력한다. 송신 측을 실행하지 않은 상태의 타임아웃은 입력이 없는 대기 결과로 구분한다.

```c
#include "ece2_474.h"
#include "main.h"
#define SPI_NSS_RX_ece2_Pin       GPIO_PIN_4
#define SPI_NSS_RX_ece2_GPIO_Port GPIOA
/* 주변장치 변수의 extern 선언은 ece2_474.h에 있습니다. */

/* UART 인터럽트 수신 실습 때 필요한 버퍼의 주석을 해제합니다.
 * 파일 범위에 두면 수신 완료 함수에서도 같은 버퍼를 사용할 수 있습니다.
 */
// static uint8_t uart2_rx_ece2;
// static uint8_t uart3_rx_ece2;

/* 주변장치 초기화가 끝난 뒤, 메인 루프에 들어가기 전에 한 번 호출됩니다. */
void ece2_474_init(void)
{
    if (HAL_SPI_DeInit(hspi: &hspi1) != HAL_OK)
        Error_Handler();

    hspi1.Init.Mode = SPI_MODE_SLAVE;
    hspi1.Init.NSS = SPI_NSS_HARD_INPUT;

    if (HAL_SPI_Init(hspi: &hspi1) != HAL_OK)
        Error_Handler();

    GPIO_InitTypeDef gpio = {0};
    gpio.Pin = SPI_NSS_RX_ece2_Pin;
    gpio.Mode = GPIO_MODE_AF_PP;
    gpio.Pull = GPIO_PULLUP;
    gpio.Speed = GPIO_SPEED_FREQ_LOW;
    gpio.Alternate = GPIO_AF5_SPI1;

    HAL_GPIO_Init(GPIOx: SPI_NSS_RX_ece2_GPIO_Port, GPIO_Init: &gpio);
}
```

그림 3: STM32 과제 9 수신 초기화 코드

예상 결과

송신값	수신 MCU 예상 출력
0x55	RX=55
0xA5	RX=A5
0x00	RX=00
0xFF	RX=FF

사전 검증 결과

슬레이브 초기화와 수신 처리를 포함한 프로그램의 빌드 종료 코드를 확인하였고, 과제8의 송신 프로그램과 과제9의 수신 프로그램은 이후 FPGA 과제에서도 사용한다. 실험실에서는 수신 측과 송신 측을 둘 다 실행한다. 역할 교대 시 CS도 새 마스터의 PB6에서 새 슬레이브의 PA4로 바꾸어 연결한다.

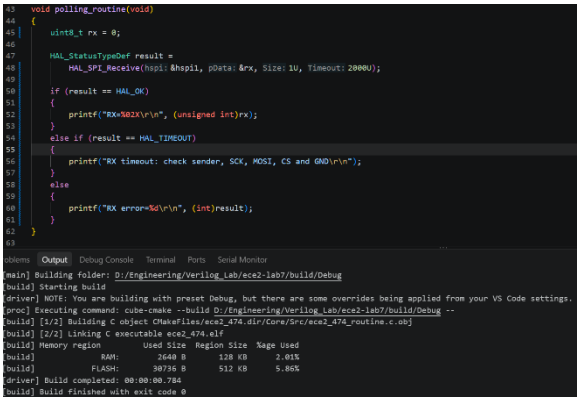


그림 4: STM32 과제 9 수신 코드 및 CMake 빌드 결과

링크: 과제9 소스 코드

링크: 과제9 빌드 및 핵심 코드 증빙

6. STM32 과제 10

설계 목표

MCU를 마스터, FPGA를 슬레이브로 연결한다. FPGA에서 SPI로 받은 바이트를 UART로 PC에 출력하고 MCU 송신값과 비교한다. 수신 회로 단독 검증 후 UART 출력까지 연결하여 검증한다.

핵심 설계

spi_slave_rx는 외부 SCK·MOSI·CS를 각각 2단 레지스터로 동기화한다. 동기화된 SCK의 현재 값과 이전 값으로 상승 에지를 검출하고, CS가 LOW인 동안 MOSI를 시프트 레지스터에 저장한다.

8번째 비트를 받으면 마지막 입력 비트까지 포함한 값을 rx_data에 저장하고 rx_valid를 한 내부 클록 동안 발생시킨다. CS가 HIGH가 되면 시프트 레지스터와 비트 카운터를 초기화하여 미완성 바이트가 다음 전송에 섞이지 않도록 한다.

```
(* ASYNC_REG = "TRUE" *) reg [1:0] sck_sync;
(* ASYNC_REG = "TRUE" *) reg [1:0] mosi_sync;
(* ASYNC_REG = "TRUE" *) reg [1:0] cs_sync;

reg       sck_previous;
reg [7:0] shift_reg;
reg [2:0] bit_count;

wire sck_rising;
assign sck_rising = sck_sync[1] && !sck_previous;
```

그림 5: STM32 과제 10 SPI 수신 핵심 코드1

```
end else begin
    sck_sync      <= {sck_sync[0], spi_sck};
    mosi_sync     <= {mosi_sync[0], spi_mosi};
    cs_sync       <= {cs_sync[0], spi_cs_n};
    sck_previous <= sck_sync[1];

    // 수신 완료 신호는 한 클록 동안만 출력
    rx_valid <= 1'b0;

    if (cs_sync[1]) begin
        // CS 해제 시 미완성 바이트를 폐기
        shift_reg <= 8'h00;
        bit_count <= 3'd0;
    end else if (sck_rising) begin
        shift_reg <= {shift_reg[6:0], mosi_sync[1]};

        if (bit_count == 3'd7) begin
            rx_data <= {shift_reg[6:0], mosi_sync[1]};
            rx_valid <= 1'b1;
            bit_count <= 3'd0;
        end else begin
            bit_count <= bit_count + 1'b1;
        end
```

그림 6: STM32 과제 10 SPI 수신 핵심 코드2

lab7_spi_rx_uart는 수신값을 LED에 반영하고 상위·하위 4비트를 각각 대문자 16진수 문자로 변환한다. 이후 CR(0x0D), LF(0x0A)를 붙여 UART로 전송한다. 예를 들어 0xA5를 받으면 PC에는 A5와 줄바꿈이 표시된다.

```
spi_slave_rx u_spi_rx (
    .clk_50mhz(clk_50mhz),
    .rst_p(rst_p),
    .spi_sck(spi_sck),
    .spi_mosi(spi_mosi),
    .spi_cs_n(spi_cs_n),
    .rx_data(rx_data),
    .rx_valid(rx_valid)
);

uart_tx #(.DIV(UART_DIV)) u_uart_tx (
    .clk(clk_50mhz),
    .rst_p(rst_p),
    .valid(tx_valid),
    .data(tx_data),
    .ready(tx_ready),
    .tx(uart_txd)
);

function [7:0] hex_ascii;
    input [3:0] value;
    begin
        if (value < 4'd10)
            hex_ascii = 8'h30 + {4'b0000, value};
        else
            hex_ascii = 8'h37 + {4'b0000, value};
    end
endfunction
```

그림 7: STM32 과제 10 UART 출력 핵심 코드1

```
    end else begin
        tx_valid <= 1'b0;

        if (rx_valid) begin
            led <= rx_data;

            if (!sending && !tx_valid) begin
                saved_data <= rx_data;
                char_index <= 2'd0;
                sending    <= 1'b1;
            end
        end

        if (sending && tx_ready && !tx_valid) begin
            case (char_index)
                2'd0: tx_data <= hex_ascii(saved_data[7:4]);
                2'd1: tx_data <= hex_ascii(saved_data[3:0]);
                2'd2: tx_data <= 8'h0D;
                2'd3: tx_data <= 8'h0A;
            endcase

            tx_valid <= 1'b1;

            if (char_index == 2'd3)
                sending <= 1'b0;
            else
                char_index <= char_index + 1'b1;
        end
    end
end
```

그림 8: STM32 과제 10 UART 출력 핵심 코드2

예상 결과

SPI 입력	예상 UART 바이트	예상 LED 버스
0x55	35 35 0D 0A	0x55
0xA5	41 35 0D 0A	0xA5
0x00	30 30 0D 0A	0x00
0xFF	46 46 0D 0A	0xFF

테스트벤치 검증 방법

수신 단독 TB는 781.25 kHz의 SCK로 0x55, 0xA5, 0x00, 0xFF를 전송한다. 추가로 CS가 HIGH인 동안의 신호 변화, 3비트 수신 후 CS 해제, 이후 0x3C의 정상 수신을 검사한다. 0x3C는 미완성 전송 이후 복구 확인을 위해 추가한 TB 입력이다.

UART 통합 TB는 앞의 네 바이트를 입력하고 UART 출력을 독립적으로 복원한다. 각 바이트에 대한 두 자리 16진수와 CR·LF, 총 16문자가 일치하는지 검사한다. 이 TB에서는 실행 시간을 줄이기 위해 UART 분주 값을 16으로 설정하였다.

사전 검증 결과

수신 단독 TB에서 LAB7_SPI_SLAVE_RX_PASS checks=5를 확인하였다.

```
RX CHECK 1: 55
RX CHECK 2: a5
RX CHECK 3: 00
RX CHECK 4: ff
RX CHECK 5: 3c
LAB7_SPI_SLAVE_RX_PASS checks=5
sim/tb_spi_slave_rx.sv:129: $finish called at 68040000 (1ps)
```

그림 9: STM32 과제 10 SPI 수신 Verilog PASS 결과

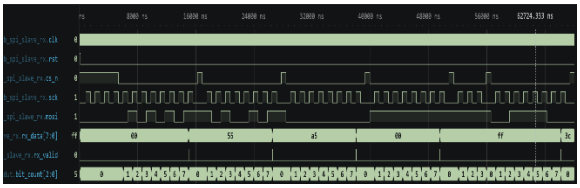


그림 10: STM32 과제 10 SPI 수신 Verilog 파형 결과

파형에서 rx_data가 55, A5, 00, FF, 3C로 갱신되고 완성된 바이트마다 rx_valid 펄스가 발생한다. 중간에 끊긴 전송은 수신 완료로 처리되지 않으며, CS 해제 후 다음 바이트가 정상 복원된다.

UART 통합 TB에서는 LAB7_SPI_RX_UART_PASS bytes=4 chars=16을 확인하였다.

```
UART LINE 1 OK: SPI=55
UART LINE 2 OK: SPI=a5
UART LINE 3 OK: SPI=00
UART LINE 4 OK: SPI=ff
LAB7_SPI_RX_UART_PASS bytes=4 chars=16
sim/tb_spi_rx_uart.sv:124: $finish called at 94050000 (1ps)
```

그림 11: SPI 수신 UART 출력 통합 Verilog PASS 결과



그림 12: SPI 수신 UART 출력 통합 Verilog 파형 결과

수신 완료 뒤 tx_data에 각 16진수 문자와 0D·0A가 순서대로 전달되고 uart_txd에 직렬 프레임이 나타난다. LED 버스도 SPI 수신값과 동일하게 변하여 수신·문자 변환·UART 출력의 연결을 확인하였다.

링크: 과제10 소스 코드

링크: FPGA SPI 수신 모듈

링크: FPGA 수신 및 UART 통합 Top

링크: 과제10 증빙자료

7. STM32 과제 11

설계 목표

FPGA를 마스터로 구성하여 1초마다 한 바이트를 전송한다. SCK를 1 MHz 이하로 만들고 오실로스코프로 파형을 확인한 뒤, 슬레이브 MCU의 VCP UART 출력과 송신값을 비교한다

핵심 설계

spi_master_tx는 대기, 비트 전송, CS 유지 상태로 동작한

다. 송신 요청 시 데이터를 저장하고 CS를 LOW로 내리면서 최상위 비트를 MOSI에 먼저 출력한다. SCK 상승 에지에서는 현재 비트를 유지하고 하강 에지에서 다음 비트로 바꾼다.

마지막 비트의 전송이 끝나면 SCK를 LOW로 유지하고 CS를 일정 시간 더 유지한 뒤 해제한다. 이후 busy를 내리고 done을 한 클록 동안 발생시킨다.

```
    IDLE: begin
        spi_sck  <= 1'b0;
        spi_cs_n <= 1'b1;
        busy     <= 1'b0;

        if (start) begin
            tx_latched <= tx_data;
            bit_index  <= 3'd7;
            half_count <= 0;
            spi_mosi   <= tx_data[7];
            spi_cs_n   <= 1'b0;
            busy       <= 1'b1;
            state      <= TRANSFER;
        end
    end

    TRANSFER: begin
        if (half_count == HALF_CYCLES - 1) begin
            half_count <= 0;

            if (!spi_sck) begin
                // 상승 에지: 수신 측이 현재 비트를 읽음
                spi_sck <= 1'b1;
            end else begin
                // 하강 에지: 다음 비트를 준비
                spi_sck <= 1'b0;

                if (bit_index == 3'd0) begin
                    state <= CS_HOLD;
                end else begin
                    bit_index <= bit_index - 1'b1;
                    spi_mosi <=
                        tx_latched[bit_index - 1'b1];
                end
            end
        end else begin
            half_count <= half_count + 1'b1;
        end
    end
```

그림 13: STM32 과제 11 SPI 송신 핵심 코드1

```
CS_HOLD: begin
    // 마지막 하강 에지 뒤에도 CS를 잠시 유지
    if (half_count == HALF_CYCLES - 1) begin
        half_count <= 0;
        spi_cs_n   <= 1'b1;
        busy       <= 1'b0;
        done       <= 1'b1;
        state      <= IDLE;
    end else begin
        half_count <= half_count + 1'b1;
    end
end
```

그림 14: STM32 과제 11 SPI 송신 핵심 코드2

50MHz 입력에서 반주기 계수 HALF_CYCLE=25를 사용하므로 SCK 주파수는 50M/(2+25) = 1MHz이다. 최상위 lab7_spi_master는 50MHz 클록마다 송신을 요청한다. 기본 송신 값은 0x55이며 전송 완료 후 LED에 같은 값을 반영한다.

```
always @(posedge clk_50mhz) begin
    if (rst_p) begin
        interval_count <= 0;
        start <= 1'b0;
        led <= 8'h00;
    end else begin
        start <= 1'b0;

        if (interval_count == SEND_INTERVAL_CYCLES - 1) begi
            interval_count <= 0;

            if (!busy)
                start <= 1'b1;
        end else begin
            interval_count <= interval_count + 1'b1;
        end

        if (done)
            led <= TX_BYTE;
    end
end
```

그림 15: STM32 과제 11 송신 주기 제어 코드

예상 결과

확인 항목	예상 결과
SCK 대기 상태	LOW
SCK 주파수	1 MHz
한 프레임의 상승 에지	8회
기본 송신값	0x55
수신 MCU의 출력	RX=55

테스트벤치 검증 방법

송신 간격을 2000클록, 즉 40 μs로 줄여 세 프레임을 검사한다. SCK 반주기는 보드 설정과 같은 500 ns이며, 한 프레임의 상승 에지 수가 8회이고 복원값이 0x55인지 확인한다.

SCK가 HIGH일 때 MOSI가 유지되는지, 첫 비트가 상승 에지 전에 준비되는지, 마지막 하강 에지 후 CS가 500 ns 유지되는지도 검사한다. 송신 완료 후 LED와 리셋 동작을 함께 확인한다.

사전 검증 결과

LAB7_SPI_MASTER_PASS frames=3을 확인하였다.

```
MASTER FRAME 1 OK: data=55 bits=8
MASTER FRAME 2 OK: data=55 bits=8
MASTER FRAME 3 OK: data=55 bits=8
LAB7_SPI_MASTER_PASS frames=3
sim/tb_spi_master.sv:140: $finish called at 128720000 (1ps)
```

그림 16: STM32 과제 11 SPI 송신 Verilog PASS 결과

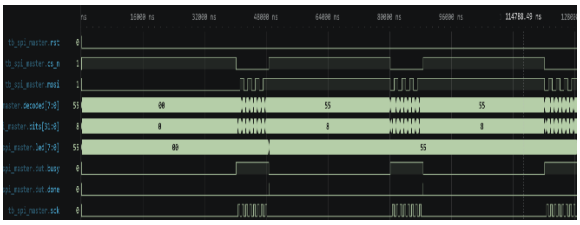


그림 17: STM32 과제 11 SPI 송신 Verilog 파형 결과

CS가 LOW인 각 전송 구간에서 8비트가 출력되고, 복원 값은 0x55이다. 전송 완료 시 done이 발생하며 LED는 0x55로 갱신된다. 따라서 예상한 결과와 파형의 형태가 동일함을 알 수 있다.

링크: 과제11 소스 코드

링크: FPGA SPI 송신 모듈

링크: FPGA 송신 Top

링크: 과제11 증빙자료

8. STM32 과제 12

설계 목표

지정된 조의 FPGA 두 대를 연결하여 SPI 송신값과 수신 FPGA의 UART 출력이 일치하는지 확인한다. 과제11의 마스터 송신 회로와 과제10의 슬레이브 수신 회로를 사용하고 역할을 교대한다.

회로 구성 및 검증 방법

송신 Top의 SCK·MOSI·CS 출력을 수신 Top의 대응 입력에 연결한다. 수신 결과는 LED와 UART로 확인하며, 별도의 SPI 회로를 추가로 작성하지 않고 앞의 두 설계를 재사용한다.

연결 TB에서는 두 Top을 서로 위상이 다른 50 MHz 클록으로 구동한다. 송신 간격은 시뮬레이션 시간을 줄이기 위해 6 ms로 설정하고 수신 UART는 기본 9600 baud를 유지한다. SPI로 0x55 두 바이트를 전달하고 UART에서 55, CR, LF가 두 번 출력되는지 검사한다.

예상 결과

입력 및 조건	예상 결과
SPI 0x55 두 번 전송	수신 데이터 0x55 두 번
UART 출력	55와 줄 바꿈 두 번
UART 문자 수	총 8바이트
전송 완료 후 송수신 LED	모두 0x55

사전 검증 결과

LAB7_SPI_LINK_PASS bytes=2 chars=8 UART=9600을 확인하였다.

```
VCD info: dumpfile sim/tb_spi_link.vcd opened for output.
LINK LINE 1 OK: 55 + CR + LF
LINK LINE 2 OK: 55 + CR + LF
LAB7_SPI_LINK_PASS bytes=2 chars=8 UART=9600
sim/tb_spi_link.sv:137: $finish called at 16226317000 (1ps)
```

그림 18: 과제 12 FPGA 송수신 연결 Verilog PASS 결과

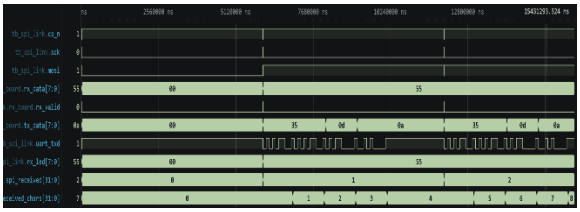


그림 19: 과제 12 FPGA 송수신 연결 Verilog 파형 결과

송신 프레임에 따라 수신 완료 신호가 발생하고 수신값이 0x55로 갱신된다. 이후 수신 FPGA의 UART에서 총 8바이트가 복원되어 두 번의 SPI 전송이 PC 출력까지 전달됨을 확인하였다. 예상결과와 동일하였고, 실제 실험 결과는 실습 중에 확인하겠다.

링크: 과제12 송수신 연결 테스트벤치

링크: 공통 FPGA 송수신 소스

링크: 과제12 증빙자료

9. 사전 검증 결과 및 문제 해결

사전 검증 결과 요약

실습	확인 내용	확인 결과
MCU 송신	1초 간격 송신 코드 컴파일, 링크	빌드 성공
MCU 수신	슬레이브 초기화 / 수신 코드 컴파일, 링크	빌드 성공
FPGA 수신 단독	5바이트 수신, CS 비활성, 중도 해제 처리	TB PASS
FPGA 수신 UART	4바이트 수신 및 UART 16바이트 출력	TB PASS
FPGA 송신	3프레임, 비트 수,클록,데이터 타이밍	TB PASS
FPGA 송수신 연결	SPI 2 바이트 및 UART 8 바이트 출력	TB PASS

검증 시 유의 사항

UART 통합 TB는 UART 분주값을 줄였고, 송신·연결 TB는 반복 전송 간격을 줄였다. 따라서 TB 파형의 시간을 그대로 실제 보드의 동작 시간으로 해석하지 않는다.

수신 UART 출력부는 1초마다 한 바이트를 받는 실습 조건을 대상으로 한다. 출력 중 들어오는 모든 바이트를 저장하는 큐는 구현하지 않았으므로 고속 연속 입력에 대한 무손실 수신까지 검증한 것은 아니다.

10. 실험 당일 검증 계획

STM32는 사용할 송신·수신 코드를 실제 빌드 대상에 반영

하고 보드에 다운로드한 뒤 실행한다. 수신 측을 먼저 실행하고 송신 측을 실행하며, Serial Monitor의 COM 포트와 9600 baud·8N1 설정을 확인한다.

FPGA는 송신·수신 설계에 맞는 RTL과 XDC를 등록하고 해당 회로를 Design Top으로 지정한다. 합성·구현 후 DRC와 타이밍 결과를 확인하고 비트스트림을 생성하여 보드에 다운로드한다. 실제 사용한 비트스트림과 구현·보드 증빙은 실험 후 정리한다.

실습	실험 당일 확인할 항목
과제8	55, A5, 00, FF의 실제 비트 순서, 상승 에지 8회, SCK 주파수, CS 구간
과제9	MCU 송신 값과 수신 VCP 출력 일치, 송수신 역할 교대
과제10	자기 조 MCU의 송신 값과 FPGA UART 출력 일치
과제11	FPGA 송신 파형과 1초 간격 확인후 MCU RX 출력 비교
과제12	지정 조 FPGA 연결, 수신 FPGA UART 출력 및 역할 교대

11. 결론

STM32의 SPI 송신·수신 프로그램을 준비하고 CMake 빌드를 완료하였다. FPGA에서는 Mode 0에 맞는 송신·수신 회로를 작성하고, 수신한 바이트를 두 자리 16진수 문자로 변환하여 UART로 출력하도록 구성하였다.

사전 시뮬레이션에서는 수신 데이터와 비트 순서, CS 해제 시 미완성 데이터 처리, 송신 클록과 UART 출력을 검사하였다. 수신 단독, 수신·UART 통합, 송신, 송수신 연결의 네 테스트벤치에서 모두 PASS를 확인하였다. 실험에서는 오실로스코프로 SCK·MOSI·CS를 측정하고, 수신 측 UART 출력과 송신값을 비교하여 실제 보드에서도 예상한 동작이 나타나는지 확인할 예정이다.

참고 문헌

[1] M. Morris Mano and Michael D. Ciletti, *Digital Design: With an Introduction to the Verilog HDL*, 5th ed., Pearson, 2013, Chapter 6, “Registers and Counters.” ISBN: 978-0-13-277420-8.
https://www.pearsonhighered.com/assets/preface/0/1/3/2/0132774208.pdf